

Table des matières

0.1	Définition	1
0.1.1	Fonction booléenne	1
0.1.2	Forme normal algébrique et degré	2
0.1.3	Code de Reed-Muller	2
0.1.4	Non linéarité	3
0.1.5	Rayon de recouvrement	3
0.2	Problématique	3
0.2.1	Inégalité de ρ	4
0.3	calcul de distance	5
0.3.1	énumération	5
0.4	Travail sur les bent	5
0.5	Algo	6
0.5.1	Algo probabiliste	6
0.5.2	Algo Tarvernier like	6
0.5.3	Algo FTL	7
[1]		

0.1 Définition

0.1.1 Fonction booléenne

Une fonction booléenne en m variables est une fonction qui prend en paramètre un m -uplet binaire et qui a ses valeurs dans $\{0, 1\}$.

$$f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2,$$

on note $B(m)$ l'ensemble des fonctions booléennes en m variables. Soit $f \in B(m)$, on définit le poids de f comme le nombre de fois où la fonction prend la valeur 1.

$$\text{wt}(f) = \#\{f(x) = 1 \mid x \in \mathbb{F}_2^m\},$$

On peut représenter une fonction par la table de ses valeurs (Truth Table) :

$$\text{TT}(f) := [f(0), f(1), \dots, f(2^m - 1)]$$

La distance de Hamming de deux fonctions booléennes f et g est définie par le nombre de fois où les fonctions prennent des valeurs différentes :

$$d_H(f, g) = \#\{f(x) \neq g(x) \mid x \in \mathbb{F}_2^m\} = \text{wt}(f + g).$$

Soit C un code dont ses éléments sont un sous ensemble des fonctions booléenne en m variables et f une fonction booléenne en m variables, on définit la distance de f au code comme la distance de Hamming minimal entre f et tous les mots de C .

$$d(f, C) = \min_{c \in C} d_H(f, c) = \min_{c \in C} \text{wt}(f + c)$$

Proposition 1. Soit f et g deux fonctions booléennes en m variables

- (i) $\deg(f) = m \iff f$ est de poids impair.
- (ii) $\deg(f) < m \iff f$ est de poids pair.
- (iii) $\text{wt}(f + g) = \text{wt}(f) + \text{wt}(g) - 2\text{wt}(fg)$

0.1.2 Forme normal algébrique et degré

Toute fonction booléenne peut s'écrire comme une somme de monômes nommé ANF (algebraic normal form)

$$f(x) = \sum_{I \subseteq \{1, \dots, m\}} a_I \prod_{i \in I} x_i$$

Le degré d'une fonction est le degré maximal de ses monômes. La valuation d'une fonction est le degré minimal de ses monômes. On note $B_s^t(m)$ l'ensemble des fonctions en m variables de valuation au moins s et de degré au plus t .

$$B_s^t(m) := \{f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2 \mid \text{val}(f) \geq s \text{ et } \deg(f) \leq t\}$$

0.1.3 Code de Reed-Muller

Le code de Reed-Muller d'ordre k en m variables, $\text{RM}(k, m)$, est l'ensemble des fonctions booléennes de degré inférieur ou égale à k .

$$\text{RM}(k, m) := \{f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2 \mid \deg(f) \leq k\}$$

Remarque 1. Chaque ligne de la matrice génératrice d'un code $\text{RM}(k, m)$ correspond à la table de vérité d'un monôme de degré inférieur ou égale à k .

La dimension K de $\text{RM}(k, m)$ est :

$$\dim(\text{RM}(k, m)) = \sum_{i=0}^k \binom{m}{i} = K$$

La longueur N de $\text{RM}(k, m)$ est :

$$N = 2^m$$

Exemple pour $\text{RM}(2, 8)$:

$$K = \binom{0}{8} + \binom{1}{8} + \binom{2}{8} = 37$$

$$N = 2^8 = 256$$

0.1.4 Non linéarité

Soit un code $\text{RM}(k, m)$ et une fonction $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$, on définit la non linéarité d'ordre k de f comme sa distance au code, autrement dit à sa distance à toutes les fonctions de degré au plus k .

$$\text{NL}_k(f) = d(f, \text{RM}(k, m))$$

Le coefficient de Walsh d'une fonction f en $a \in \mathbb{F}_2^m$ est définie par :

$$\hat{f}(a) = \sum_{x \in \mathbb{F}_2^m} (-1)^{f(x) + a \cdot x}$$

La non linéarité d'ordre 1 peut aussi être définie par :

$$\text{NL}(f) = 2^{m-1} - \frac{1}{2} \max_{a \in \mathbb{F}_2^m} |\hat{f}(a)|.$$

0.1.5 Rayon de recouvrement

Le rayon de recouvrement $\rho(k, m)$ d'un code $\text{RM}(k, m)$ de Reed-Muller, est la distance maximale entre ce dernier et l'ensemble des fonctions booléennes en m variables.

$$\rho(k, m) = \max_{f \in \text{B}(m)} \text{NL}_k(f) = \max_{f \in \text{B}(m)} d(f, \text{RM}(k, m))$$

0.2 Problématique

Connaitre le rayon de recouvrement d'un code de Reed Muller est interessant pour "blablalbalbalblabla". La taille de l'espace des fonctions booléennes est immense. Pour $m = 8$ l'ensemble des fonctions booléennes en 8 variable est $2^{2^8} = 2^{256}$. De plus, si on veut calculer la non linéarité d'ordre 2 pour toutes les fonctions, calculer la distance entre une fonction de $\text{B}(8)$ et tout $\text{RM}(2, 8)$ n'est pas non plus trivial car ce code comporte 2^{37} éléments. Le tableau du récent article de Jinjie GAO [1] (voir tab 1) recense toutes les valeurs de rho connu jusqu'à $m = 10$. On peut voir que l'on connait exactement toutes les valeurs de rho jusqu'à $m = 7$. Au-delà certaines valeurs sont comprises dans un intervalle. Notament

$$88 \leq \rho(2, 8) \leq 96$$

Notre but est de trouver une fonction dont la non linéarité est plus grande que 88.

TABLE 1 – Tableau des valeur de $\rho(k, n)$

$k \backslash n$	3	4	5	6	7	8	9	10
1	2	6	12	28	56	120	242-244	496
2	1	2	6	18	40	88-96	196-216	400-460
3	0	1	2	8	20	56-60	111-156	194-372
4		0	1	2	8	26	58-86	≤ 242
5			0	1	2	10	28-36	≤ 122
6				0	1	2	10	≤ 46
7					0	1	2	12
8						0	1	2

0.2.1 Inégalité de ρ

On sait que

$$\rho(k, n) \leq \rho(k, n-1) + \rho(k-1, n-1)$$

Preuve :

Soit $x = (a, b)$ un vecteur binaire dans $\mathbb{F}_2^m$ avec a et b de longueur $2^m - 1$. Tous les mots c dans $\text{RM}(k, m)$ peuvent d'écrire comme $c = u + x_8 v$ avec $u \in \text{RM}(k, m-1)$ $v \in \text{RM}(k-1, m-1)$. Il existe un mot $u \in \text{RM}(k, m-1)$ tel que $d(a, u) \leq \rho(k, m-1)$. De plus is existe un mot $v \in \text{RM}(k-1, m-1)$ tel que $d(b+u, v) \leq \rho(k-1, m-1)$. Puisque $x = (a, b)$ et $c = (u, u+v)$ $d(c, x) = d(u, a) + d(u+v, b) = d(u, a) + d(u+b, v)$. Alors le mot c vérifie $d(c, x) \leq \rho(k, m-1) + \rho(k-1, m-1)$.

Pour $\text{RM}(2, 8)$ on voudrait savoir si il existe une fonction f qui atteindrait cette borne supérieure tel que $\text{NL}_2(f) = 96$.

Décomposition de Plotkin

Toute fonction booléenne f en m variable de degré k autrement dit $f \in \text{RM}(k, m)$ peut s'écrire sous la forme

$$f = x_m g + h$$

avec $g \in \text{RM}(k-1, m-1)$ et $h \in \text{RM}(k, m-1)$ si $x_m = 0, f = h$ et si $x_m = 1, f = g + h$ la table de vérité de toute fonction est donc de la forme

$$[h \mid h + g]$$

Ce qui veut dire que pour que $\text{NL}_2(f) = 96$ il faut que pour n'importe quelle fonction $t = x_8 v + u \in \text{RM}(2, 8)$

$$\text{wt}(t + f) \geq 96 \equiv \text{wt}(h + v) + \text{wt}(h + v + g + u) \geq 96$$

0.3 calcul de distance

0.3.1 énumération

Dans ce paragraphe je vais présenter une methode pour calculer la distance d'une fonction f à un code C . la manière la plus simple est consiste à énumérer tous les mots du code. Pour cela il faut générer toutes les combinaisons linéaires des lignes de la matrice génératrice G du code. La matrice G comporte k ligne $l_1, l_2, \dots, l_k$ donc chaque mot c du code s'écrit

$$c = \sum_{i=1}^k u_i l_i \quad \text{où} \quad u \in \mathbb{F}_2^k \quad u = (u_1, u_2, \dots, u_k)$$

Nous avons donc besoin d'énumérer tous les u mais au lieu d'utilisé l'ordre binaire classique $0 = (000), 1 = (001), 2 = (010) \dots$ on utilise le code de Gray afin de changé un bit à la fois, l'indice de ce bit correspond à l'indice de la ligne à additionner afin de calculer le prochain mot du code. Par exemple pour $k = 3$:

$G = \begin{bmatrix} l_1 \\ l_2 \\ l_3 \end{bmatrix}$	u	000	001	011	010	110	111	101	100
	ligne additionné	0	l_1	l_2	l_1	l_3	l_1	l_2	l_1
	c	0	l_1	$l_1 + l_2$	l_2	$l_3 + l_2$	$l_3 + l_2 + l_1$	$l_3 + l_1$	l_3

Dans l'exemple pour passer du mot $l_1 + l_2$ représenté par le vecteur $u = 011$ on passe à la prochaine valeur de u dans le code de Gray qui est $u = 010$, le bit d'indice 1 a été modifié donc pour passer au prochain mot il suffit de faire $l_1 + l_2 + l_1 = l_2$.

0.4 Travail sur les bent

Une fonction booléenne f est dite bent si sa distance aux fonctions affines est la plus grande possible $NL_1(f) = \rho(1, m)$ nous avons voulu étudier la non linéarité d'ordre 2 pour les bent en 8 variables. Calculer la non linéarité d'ordre 2 de toutes les fonction bent aurait été impossible étant donnée qu'il y en a 190 million et 2^{37} mot dans $RM(2, 8)$. Il nous fallait donc trouver un moyen d'estimer rapidement la non linéarité d'une fonction à l'aide d'un algorithme probabiliste. La classe latérale d'une fonction f modulo C est l'ensemble des translaté de f par les éléments de C

$$f + C = \{f + c \mid c \in C\}$$

Calculé la distance de f à un code revient donc à trouvé le poid minimal de sa classe latérale. L'algorithme probabiliste que l'on a utilisé a pour but de creer un translaté aléatoire de f de poid au plus $m - k$ avec m et k la longueur et la dimension du code. Cette algorithme a été très rapide et efficace pour trouver et éliminer les fonctions de faible non linéarité ce qui nous a permis de traiter l'intégralité des fonctions bent en quelque heure. Nous avons trouvé que la non linéarité d'ordre 2 des fonctions bent est de 88. On pourrait dire par intuition que $\rho(2, 8) = 88$ mais rien est sûr.

0.5 Algo

0.5.1 Algo probabiliste

voir Algo 1

Algorithm 1 Random translate

Require: G : Une matrice génératrice d'un code $RM(k, m)$ de dimension $[K, M]$

f : Une fonction booléenne en n variable représentée par sa table de vérité de longueur N

```
1: for  $i \leftarrow 0$  to  $K - 1$  do
2:   repeat
3:      $pivot \leftarrow \text{rand}() \bmod M$ 
4:   until  $G[i][pivot] = 1$ 
5:   for  $j \leftarrow i + 1$  to  $K - 1$  do
6:     if  $j \neq i$  and  $G[j][pivot] = 1$  then
7:        $G[j] \leftarrow G[j] + G[i]$ 
8:     end if
9:   end for
10:  if  $f[pivot] = 1$  then
11:     $f \leftarrow f + G[i]$ 
12:  end if
13: end for
14: return  $f$ 
```

0.5.2 Algo Tarvernier like

Voir Algo 2

Algorithm 2 Calcule distance

Require: Une fonction booléenne en n variable : $f \in B(m) = x_n g + h$

L'ordre de non linéarité recherché : k

M et K la longueur et la dimension du code $RM(k, m)$

```
1:  $best \leftarrow M$ 
2: for  $u \in RM(k - 1, m - 1)$  do
3:   if  $\text{wt}(g + u) < best$  then
4:     for  $v \in RM(k, m - 1,)$  do
5:        $best \leftarrow \min \text{wt}(h + v) + \text{wt}(h + v + g + u)$ 
6:     end for
7:   end if
8: end for
9: return  $best$ 
```

0.5.3 Algo FTL

```

1: score  $\leftarrow 0$ 
2: for  $P \in B_2^2(7)$  do
3:    $\gamma_0 \leftarrow \max_{a \in \mathbb{F}_2^7} |\widehat{f_0 + P}(a)|$ 
4:    $\gamma_1 \leftarrow \max_{a \in \mathbb{F}_2^7} |\widehat{f_1 + P}(a)|$ 
5:   if  $\gamma_0 + \gamma_1 \geq$  then
6:     for  $l \in B_1^1(7)$  do
7:        $h \leftarrow f + P + x_8 l$ 
8:        $\gamma \leftarrow \max_{a \in \mathbb{F}_2^8} |\widehat{h}(a)|$ 
9:       if  $\gamma >$  score then
10:        score  $\leftarrow \gamma$ 
11:       end if
12:     end for
13:   end if
14: end for
15: return score

```

Bibliographie

- [1] Jinjie GAO. “Numerical Results and Asymptotic Lower Bound on the Covering Radius of Reed-Muller Codes $RM(2,11)$ and $RM(3,n)$ ”. In : *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* 109.1 (2026), p. 35-46. DOI : 10.1587/TRANSFUN.2025EAP1015. URL : <https://doi.org/10.1587/transfun.2025eap1015>.